LAPORAN PRATIKUM PEKAN 4
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Queue*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan Queue.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 29 April 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN 4

1.1 Latar Belakang 4

1.2 Tujuan 4

1.3 Manfaat 4

BAB II PEMBAHASAN 5

2.1 Iterasi Queue 5

2.1.1 Kode Program 5

2.1.2 Uraian Kode Program 5

2.1.3 Output 6

2.2 Queue - Array 7

2.2.1 Kode Program 7

2.2.2 Uraian Kode Program 7

2.3 Queue – ArrayDriver 8

2.3.1 Kode Program 8

2.3.2 Uraian Kode Program 8

2.3.3 Output 9

2.4 Queue – ArrayLinkedList 9

2.4.1 Kode Program 9

2.4.2 Uraian Kode Program 9

2.4.3 Output 10

2.5 Queue – ReverseData 11

2.5.1 Kode Program 11

2.5.2 Uraian Kode Program 11

2.5.3 Output 12

BAB III TUGAS 13

3.1 Main Class 13

3.1.1 Kode Program 13

3.1.2 Uraian Kode Program 14

3.2 Driver Class 16

3.2.1 Kode Program 16

3.2.2 Uraian Kode Program 16

3.3 Output 17

BAB III KESIMPULAN 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman karena berperan dalam mengatur dan mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah *Queue* (antrian), yaitu struktur data yang menerapkan prinsip FIFO (*First In First Out*), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian di kasir, sistem pencetakan dokumen, dan pengelolaan proses dalam sistem operasi.

Dalam pemrograman, implementasi Queue sangat penting untuk memahami bagaimana data diproses secara berurutan dan terkontrol. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep dasar Queue serta mampu mengimplementasikannya dalam bahasa pemrograman.

1.2 Tujuan

1. Memahami konsep dasar struktur data Queue.
2. Mampu mengimplementasikan Queue dalam pemrograman.
3. Melatih kemampuan logika pemrograman dalam mengelola data secara terstruktur.

1.3 Manfaat

1. Meningkatkan pemahaman tentang struktur data, khususnya Queue.
2. Membantu mahasiswa dalam mengembangkan kemampuan pemrograman yang lebih terstruktur.
3. Meningkatkan kemampuan analisis dan pemecahan masalah dalam pemrograman.

BAB II
PEMBAHASAN

2.1 Iterasi Queue

2.1.1 Kode Program

```java
package Pekan4_2511533002;
import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;
public class IterasiQueue_2511533002 {
    public static void main(String[] args) {

        Queue<String> q_3002 = new LinkedList<> () ;

        q_3002.add("Praktikum");
        q_3002.add("Struktur");
        q_3002.add("Data");
        q_3002.add("Dan");
        q_3002.add("Algoritma");
        Iterator<String> iterator = q_3002.iterator() ;
        while ( iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }

}
```

2.1.2 Uraian Kode Program

1. **Inisialisasi Queue**
   - **Queue<String> q_3002 = new LinkedList<>();**
     Baris ini membuat sebuah objek Queue bernama q_3002 yang secara spesifik hanya bisa menyimpan tipe data teks, program ini menggunakan LinkedList sebagai fondasinya.
2. **Memasukkan Data**
   - **q_3002.add("Praktikum");**
     q_3002.add("Struktur");
     q_3002.add("Data");
     q_3002.add("Dan");
     q_3002.add("Algoritma");
     Sistem menambahkan lima kata ke dalam memori antrean menggunakan method .add(). Data yang pertama kali dimasukkan ("Praktikum") akan berada di posisi paling depan, dan data yang terakhir dimasukkan ("Algoritma") akan berada di posisi paling belakang antrean.
3. **Proses Iterasi**
   - **q_3002.iterator()**

Membuat sebuah kursor (bernama iterator) yang disiapkan untuk menunjuk ke elemen-elemen di dalam antrean q_3002.

- **iterator.hasNext()**
  Kondisi dalam blok while yang akan mengecek Apakah masih ada data selanjutnya di dalam antrean ini, jika masih ada, proses akan dilanjutkan.
- **iterator.next**()
  Mengambil nilai elemen yang sedang ditunjuk saat itu, kemudian menggeser kursor ke elemen berikutnya.

2.1.3 Output

```
Praktikum Struktur Data Dan Algoritma
```

2.2 Queue - Array

2.2.1 Kode Program

```java
package Pekan4_2511533002;

public class QueueArray_2511533002 {
    int front_3002,rear_3002,size_3002;
    int capacity_3002;
    int array_3002[];

    public QueueArray_2511533002(int capacity) {
        this.capacity_3002 = capacity;
        front_3002 = this.size_3002 = 0;
        rear_3002 = capacity - 1;
        array_3002 = new int [this.capacity_3002];

    }

    boolean isFull_2511533002 (QueueArray_2511533002 queue) {
        return (queue.size_3002 == queue.capacity_3002) ;

    }

    boolean isEmpty_2511533002 (QueueArray_2511533002 queue) {
        return (queue.size_3002 == 0) ;

    }

    void enqueue_2511533002 ( int item ) {
        if (isFull_2511533002 (this))
            return ;
        this.rear_3002 = (this.rear_3002 + 1 ) % this.capacity_3002;
        this.array_3002 [this.rear_3002] = item;
        this.size_3002 = this.size_3002 + 1;
        System.out.println(item + " enqueued to queue");
    }
    int dequeue_3002 () {
        if (isEmpty_2511533002 (this))
            return Integer.MIN_VALUE;
        int item = this.array_3002[this.front_3002];
        this.front_3002 = (this.front_3002 + 1 )  % this.capacity_3002;
        this.size_3002 = this.size_3002 - 1;
        return item;
    }
    int front_3002() {
        if(isEmpty_2511533002(this) )
        return Integer.MIN_VALUE;

        return this.array_3002[this.front_3002];

    }
    int rear_3002() {
        if (isEmpty_2511533002(this))
            return Integer.MIN_VALUE;
        return this.array_3002 [this.rear_3002];

    }
    // mencetak elemen antrian
    void display_2511533002 () {
        int i_3002;
        if ( front_3002 == rear_3002) {
            System.out.println("\nAntrian kosong\n");
            return;
        }
        // kunjungi dari belakang dan cetak
        for (i_3002 = front_3002; i_3002 < rear_3002; i_3002++) {
            System.out.printf(" %d <-- " , array_3002[i_3002]);
        }
        return;
    }

}
```

2.2.2 Uraian Kode Program

1. **Deklarasi Variabel**
    - **Size_3002**
      Batas maksimal jumlah data yang bisa ditampung.
    - **Front_3002**
      Menunjuk indeks (posisi) data yang berada paling depan.
    - **Rear_3002**
      Menunjuk indeks data yang berada paling belakang.

2. **Konstruktor**
    - **public QueueArray_2511533002(int capacity)**
      Konstruktor ini akan dijalankan pertama kali saat antrean dibuat. Ia mengatur kapasitas antrean, membuat array baru sesuai kapasitas tersebut, dan mengatur nilai awal front (0) serta size (0). Uniknya, rear diatur ke capacity - 1 agar saat penambahan data pertama kali, posisinya akan memutar (melingkar) kembali ke indeks ke-0.
3. **Method utama**
    - **enqueue_2511533002(int item)**
      Jika antrean sudah penuh, method langsung berhenti (return). Jika belum, penunjuk rear akan maju satu langkah.
    - **dequeue_3002()**
      Jika antrean kosong, ia mengembalikan Integer.MIN_VALUE sebagai penanda error/kosong. Jika ada data, program akan mengambil angka dari indeks front, lalu memajukan front satu langkah ke belakang, dan mengurangi size.

2.3 Queue – ArrayDriver

2.3.1 Kode Program

```java
package Pekan4_2511533002;

public class QueueArrayDriver_2511533002 {

    public static void main(String[] args) {
        QueueArray_2511533002 queue = new QueueArray_2511533002 (1000);
        queue.enqueue_2511533002(10);
        queue.enqueue_2511533002(20);
        queue.enqueue_2511533002(30);
        queue.enqueue_2511533002(40);
        System.out.println("item di depan " + queue.front_3002 () );
        System.out.println("item di belakang " + queue.rear_3002() );
        System.out.println("tampilkan queue");
        queue.display_2511533002();
        System.out.println();
        System.out.println(queue.dequeue_3002() + " dihapus dari queue");
        System.out.println("item di depan " + queue.front_3002 () );
        System.out.println("item di belakang " + queue.rear_3002() );
        System.out.println("tampilkan queue setelah satu data dihapus");
        queue.display_2511533002();



    }

}
```

2.3.2 Uraian Kode Program

1. **Inisialisasi**
    - **QueueArray_2511533002 queue = new QueueArray_2511533002 (1000);**
      Sistem membuat sebuah objek antrean baru bernama queue. Angka 1000 di dalam kurung adalah capacity yang dikirim ke constructor di kelas sebelumnya. Artinya, Anda memesan tempat di memori untuk menampung maksimal 1000 angka.
2. **Mengeluarkan Data**
    - **System.out.println(queue.dequeue_3002() + " dihapus dari queue");**
      Method dequeue_3002() dijalankan. Karena prinsipnya FIFO, data yang paling depan (yaitu **10**) akan ditarik keluar dari antrean, nilainya dikembalikan ke perintah print, dan variabel front akan bergeser menunjuk ke angka 20.

2.3.3 Output

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
item di depan 10
item di belakang 40
tampilkan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan 20
item di belakang 40
tampilkan queue setelah satu data dihapus
 20 <--  30 <--
```

2.4 Queue – ArrayLinkedList

2.4.1 Kode Program

```java
package Pekan4_2511533002;
import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511533002 {
    public static void main(String[] args) {

        Queue<Integer> q_3002 = new LinkedList<> ();
        // menambah elemen ke dalam antrian

        for(int i = 0 ; i < 6 ; i ++)
            q_3002.add(i);

        // menampilkan ke dalam antrian
        System.out.println("elemen antrian " + q_3002);
        //untuk menghapus kepala antrian
        int hapus_1004 = q_3002.remove();
        System.out.println("elemen yang dihpaus " + hapus_1004);
        System.out.println(q_3002);
        //untuk melihat antrian di depan
        int depan = q_3002.peek();
        System.out.println("elemen terdepan " + depan);

        int banyak = q_3002.size();
        System.out.println("size antrian " + banyak);

    }

}
```

2.4.2 Uraian Kode Program

1. **Memasukkan Data Secara Looping**

- **for(int i = 0 ; i < 6 ; i ++) q_3002.add(i);**
  Sistem melakukan perulangan for sebanyak 6 kali dari i = 0 hingga i = 5. Di dalam setiap perulangan, nilai i saat itu langsung dimasukkan ke dalam antrean menggunakan perintah .add(i).

2. **Menghapus Data**
   - **int hapus_1004 = q_3002.remove();**
     System.out.println("elemen yang dihpaus " + hapus_1004);
     System.out.println(q_3002);
     - Perintah .remove() bertugas mengambil dan menghapus data yang **paling depan** (yaitu angka 0) sesuai aturan FIFO (First-In, First-Out).
     - Data yang terhapus itu diselamatkan/disimpan ke dalam variabel baru bernama hapus_1004, lalu dicetak ke layar.
     - Selanjutnya, isi antrean dicetak kembali untuk membuktikan bahwa angka 0 sudah tidak ada di dalam antrean.
3. **Mengintip Elemen Terdepan**
   - **int depan = q_3002.peek();**
     System.out.println("elemen terdepan " + depan);
     Perintah .peek() sama fungsinya dengan method front() pada kode manual Anda sebelumnya. Ia hanya digunakan untuk **melihat** data apa yang saat ini antre di barisan paling depan tanpa mengeluarkannya dari antrean. Karena angka 0 sudah dihapus, maka elemen terdepannya sekarang adalah angka 1.
4. **Mengecek Sisa Ukuran Data**
   - **int banyak = q_3002.size();**
     System.out.println("size antrian " + banyak);
     Perintah .size() menghitung jumlah total elemen yang saat ini ada di dalam antrean. Awalnya ada 6 elemen, dan 1 elemen sudah dihapus, sehingga sisanya adalah 5 elemen.

2.4.3 Output

```
elemen antrian [0, 1, 2, 3, 4, 5]
elemen yang dihpaus 0
[1, 2, 3, 4, 5]
elemen terdepan 1
size antrian 5
```

2.5 Queue – ReverseData

2.5.1 Kode Program

```java
package Pekan4_2511533002;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511533002 {
    public static void main(String[] args) {
        Queue<Integer> q_3002 = new LinkedList<Integer> ();
        q_3002.add(1);
        q_3002.add(2);
        q_3002.add(3);
        System.out.println("Sebelum reverse " + q_3002 );
        Stack<Integer> s_3002 = new Stack<Integer> () ;
        while (!q_3002.isEmpty()) {
            s_3002.push(q_3002.remove() );
        }
        while (!s_3002.isEmpty()) {
            q_3002.add(s_3002.pop());
        }


        System.out.println("data sesudah di reverse " + q_3002);
    }

}
```

2.5.2 Uraian Kode Program

1. **Memindahkan Data dari Queue ke Stack**
    - **Stack<Integer> s_3002 = new Stack<Integer> () ;**
      while (!q_3002.isEmpty()) { s_3002.push(q_3002.remove()); }
      Program membuat sebuah Stack kosong bernama s_3002. Kemudian, sebuah perulangan while memindahkan seluruh data dari antrean ke dalam tumpukan:
      - Langkah 1: remove() mengambil 1 dari depan Queue, lalu push() memasukkannya ke dasar Stack.
      - Langkah 2: remove() mengambil 2 dari depan Queue, lalu push() menumpuknya di atas angka 1.
      - Langkah 3: remove() mengambil 3 dari depan Queue, lalu push() menumpuknya di posisi paling atas Stack.
      - Kondisi saat ini: Queue kosong, dan Stack berisi tumpukan angka dengan 3 berada di paling atas.
2. **Memindahkan Data dari Stack kembali ke Queue**
    - **while (!s_3002.isEmpty()) {**
      q_3002.add(s_3002.pop()); }
      Perulangan while kedua ini memindahkan data kembali ke asalnya. Karena Stack bersifat LIFO, data yang berada di paling atas tumpukan akan diambil terlebih dahulu:
      - Langkah 1: pop() mengambil 3 dari atas Stack, lalu add() memasukkannya ke Queue yang masih kosong.

- Langkah 2: pop() mengambil 2 dari Stack, lalu add() memasukkannya ke belakang angka 3 di Queue.
- Langkah 3: pop() mengambil 1 (yang tadi di dasar tumpukan), lalu add() memasukkannya ke posisi paling belakang di Queue.

2.5.3 Output

```
Sebelum reverse [1, 2, 3]
data sesudah di reverse [3, 2, 1]
```

BAB III
TUGAS

3.1 Main Class

3.1.1 Kode Program

```java
package Pekan4_2511533002;

public class AntrianKafe_2511533002 {
    int front;
    int rear;
    int max;
    String[] queue;

    public AntrianKafe_2511533002(int kapasitas) {
        this.max = kapasitas;
        this.queue = new String[this.max];
        this.front = -1;
        this.rear = -1;
    }

    boolean isEmpty() {
        return (front == -1 || front > rear);
    }

    boolean isFull() {
        return (rear == max - 1);
    }

    void enqueue(String namaPelanggan) {
        if (isFull()) {
            System.out.println("Antrian penuh! Mohon tunggu pesanan lain selesai.");
        } else {
            if (front == -1) {
                front = 0;
            }
            rear++;
            queue[rear] = namaPelanggan;
            System.out.println("Pesanan atas nama " + namaPelanggan + " masuk ke antrian.");
        }
    }

    void dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong! Tidak ada pesanan untuk diproses.");
        } else {
            String pesananSelesai = queue[front];
            System.out.println("Pesanan atas nama " + pesananSelesai + " telah selesai dibuat.");
            front++;

            if (front > rear) {
                front = -1;
                rear = -1;
            }
        }
    }

    void display() {
        if (isEmpty()) {
            System.out.println("Antrian pesanan kosong!");
        } else {
            System.out.println("Daftar Antrian Pesanan:");
            int nomor = 1;
            for (int i = front; i <= rear; i++) {
                System.out.println(nomor + ". " + queue[i]);
                nomor++;
            }
        }
    }

    void Reverse() {
        if (isEmpty()) {
            System.out.println("Antrian kosong, tidak ada yang bisa dibalik.");
        } else {
            int i = front;
            int j = rear;
            while (i < j) {
                String temp = queue[i];
                queue[i] = queue[j];
                queue[j] = temp;
                i++;
                j--;
            }
            System.out.println("Urutan antrian pesanan berhasil dibalik.");
        }
    }
}
```

3.1.2 Uraian Kode Program

1. **Konstruktor**
   - **public AntrianKafe_2511533002(int kapasitas) {**
     this.max = kapasitas;
     this.queue = new String[this.max];
     this.front = -1;
     this.rear = -1; }
     - this : Kata kunci ini digunakan agar Java tidak bingung membedakan antara variabel milik *class* dan variabel parameter.
     - this.queue = new String[this.max]; : Di sinilah memori komputer benar-benar dipesan sesuai kapasitas yang diinput pengguna.
2. **Pengecekan**
   - **boolean isEmpty() { return (front == -1 || front > rear); }**
     boolean isFull() { return (rear == max - 1); }
     - isEmpty (Kosong): Ada dua kondisi antrean dianggap kosong:
       1. front == -1: Kondisi awal saat toko baru buka.
       2. front > rear: Kondisi di mana pelanggan masuk, lalu dilayani semua. Saat dilayani, penanda front terus maju hingga akhirnya menyalip rear. Jika front menyalip rear, artinya tidak ada lagi orang di antara mereka.
     - isFull (Penuh): Karena batas indeks maksimal *array* adalah max - 1 (contoh: jika kapasitas 5, indeks mentok di 4). Jika rear sudah mencapai indeks terakhir ini, antrean ditutup.
3. **Operasi Menambah Data**
   - **void enqueue(String namaPelanggan) {**
     if (isFull()) { ... }
     else {
     if (front == -1) { front = 0; }
     rear++;
     queue[rear] = namaPelanggan;
     - if (front == -1) { front = 0; }: Jika ini adalah pelanggan pertama (toko baru buka), penanda front yang tadinya -1 harus digeser ke 0 agar menunjuk ke orang pertama tersebut.
     - rear++: Penanda belakang digeser mundur satu langkah untuk membuka ruang.
     - queue[rear] = namaPelanggan: Nama pelanggan diletakkan di ruang baru yang ditunjuk rear tadi.
4. **Operasi Mengeluarkan Data**

```
void dequeue() {
   if (isEmpty()) { ... }
   else {
      String pesananSelesai = queue[front];
      System.out.println("Pesanan atas nama " + pesananSelesai + " telah selesai
dibuat.");
      front++;
      if (front > rear) {
         front = -1;
         rear = -1;
```

- String pesananSelesai = queue[front];: Mengambil nama orang yang berada tepat di penanda front.
- front++: Ini adalah kunci dari konsep *Queue* menggunakan *array*. Kita tidak menghapus datanya dari memori, melainkan kita sekadar memajukan penanda depannya. Data lama secara teknis masih ada di *array*, tapi sudah "diabaikan" dan dianggap tidak ada.
- Blok if (front > rear): Ini adalah perbaikan krusial untuk kelemahan Linear Queue. Jika semua pelanggan hari itu sudah dilayani (sehingga front menyalip rear), sistem me-reset front dan rear kembali ke -1. Jika ini tidak dilakukan, *array* akan terjebak dalam status isFull selamanya dan tidak bisa dipakai lagi (padahal semua orang sudah selesai dilayani).

5. **Menampilkan Data yang Relavan**
   - **for (int i = front; i <= rear; i++) {**
     System.out.println(nomor + ". " + queue[i]);
     nomor++;

     looping tidak dimulai dari 0 sampai max, melainkan dari front sampai rear. Ini membuktikan poin pada bagian dequeue tadi: meskipun ada data "sampah" di indeks sebelum front, program akan mengabaikannya dan hanya mencetak daftar pelanggan yang benar-benar masih mengantre.

3.2 Driver Class

3.2.1 Kode Program

```java
package Pekan4_2511533002;
import java.util.Scanner;

public class AntrianKafeDriver_2511533002 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Masukkan batas maksimal antrian kafe: ");
        int kapasitas = scanner.nextInt();
        scanner.nextLine();

        AntrianKafe_2511533002 kafe = new AntrianKafe_2511533002(kapasitas);
        int pilihan;

        do {
            System.out.println("\n=== SISTEM ANTRIAN PESANAN KAFE ===");
            System.out.println("1. Tambah Pesanan Baru");
            System.out.println("2. Selesaikan Pesanan");
            System.out.println("3. Lihat Daftar Antrian");
            System.out.println("4. Reverse Urutan Antrian");
            System.out.println("5. Tutup Program");
            System.out.print("Pilih menu: ");
            pilihan = scanner.nextInt();
            scanner.nextLine();
            System.out.println();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = scanner.nextLine();
                    kafe.enqueue(nama);
                    break;
                case 2:
                    kafe.dequeue();
                    break;
                case 3:
                    kafe.display();
                    break;
                case 4:
                    kafe.Reverse();
                    break;
                case 5:
                    System.out.println("Sistem ditutup. Sampai jumpa!");
                    break;
                default:
                    System.out.println("Pilihan salah, silakan masukkan angka 1-5.");
            }
        } while (pilihan != 5);

        scanner.close();
    }
}
```

3.2.2 Uraian Kode Program

1. **Sistem Perulangan (do-while)**
   Program menggunakan struktur do-while. Artinya: "Lakukan pencetakan teks menu ini ke layar secara terus-menerus, selama kasir belum menekan angka 5". Inilah yang membuat program tidak langsung mati setelah kita memasukkan satu nama pelanggan; ia akan terus mengulang menunya.
2. **Logika Pemillihan Menu (switch-case)**
   Setelah kasir mengetik angka pilihan 1 sampai 5, program masuk ke blok switch yang bertindak seperti rel kereta bercabang

3.3 Output

```
Masukkan batas maksimal antrian kafe: 2

=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 1

Masukkan nama pelanggan: Rudi
Pesanan atas nama Rudi masuk ke antrian.

=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 1

Masukkan nama pelanggan: Rendi
Pesanan atas nama Rendi masuk ke antrian.

=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 3

Daftar Antrian Pesanan:
1. Rudi
2. Rendi
```

```
=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 4

Urutan antrian pesanan berhasil dibalik.

=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 2

Pesanan atas nama Rendi telah selesai dibuat.

=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 3

Daftar Antrian Pesanan:
1. Rudi
```

```
=== SISTEM ANTRIAN PESANAN KAFE ===
1. Tambah Pesanan Baru
2. Selesaikan Pesanan
3. Lihat Daftar Antrian
4. Reverse Urutan Antrian
5. Tutup Program
Pilih menu: 5

Sistem ditutup. Sampai jumpa!
```

BAB III
KESIMPULAN

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa **Queue (antrian)** merupakan struktur data yang bekerja dengan prinsip **FIFO (First In First Out)**, yaitu data yang masuk pertama akan keluar lebih dulu.

Pada **iterasi queue**, kita dapat menelusuri dan menampilkan seluruh elemen dalam antrian tanpa mengubah urutan data, sehingga membantu dalam memahami isi queue secara keseluruhan.

Pada **reverse data**, proses pembalikan urutan queue dapat dilakukan dengan bantuan struktur data lain seperti stack. Hal ini menunjukkan bahwa kombinasi dua struktur data dapat digunakan untuk menyelesaikan permasalahan tertentu secara efisien.

Implementasi **queue menggunakan array** memperlihatkan cara kerja antrian dengan kapasitas tetap, namun memiliki keterbatasan seperti kemungkinan terjadinya overflow dan kurang fleksibel dalam penggunaan memori.

Sedangkan **queue menggunakan LinkedList** memberikan kelebihan dalam hal fleksibilitas ukuran karena bersifat dinamis, sehingga lebih efisien untuk penggunaan data yang tidak diketahui jumlahnya secara pasti.

Secara keseluruhan, praktikum ini membantu memahami konsep dasar queue, operasi-operasi yang tersedia, serta perbedaan implementasi yang dapat dipilih sesuai kebutuhan.